Lab 7: Fuzzing for Input Validation Bugs

L6_L7_38 Elevator Dispatch

elevator_dispatch.py · Hypothesis + pytest · Terminal output captured from the runs

SRN: PES1UG24CS402

NAME: SAIKAVIN S

SECTION: 5G

DATE: 11/11/2026

STEP 2: Execution of the original (buggy) application

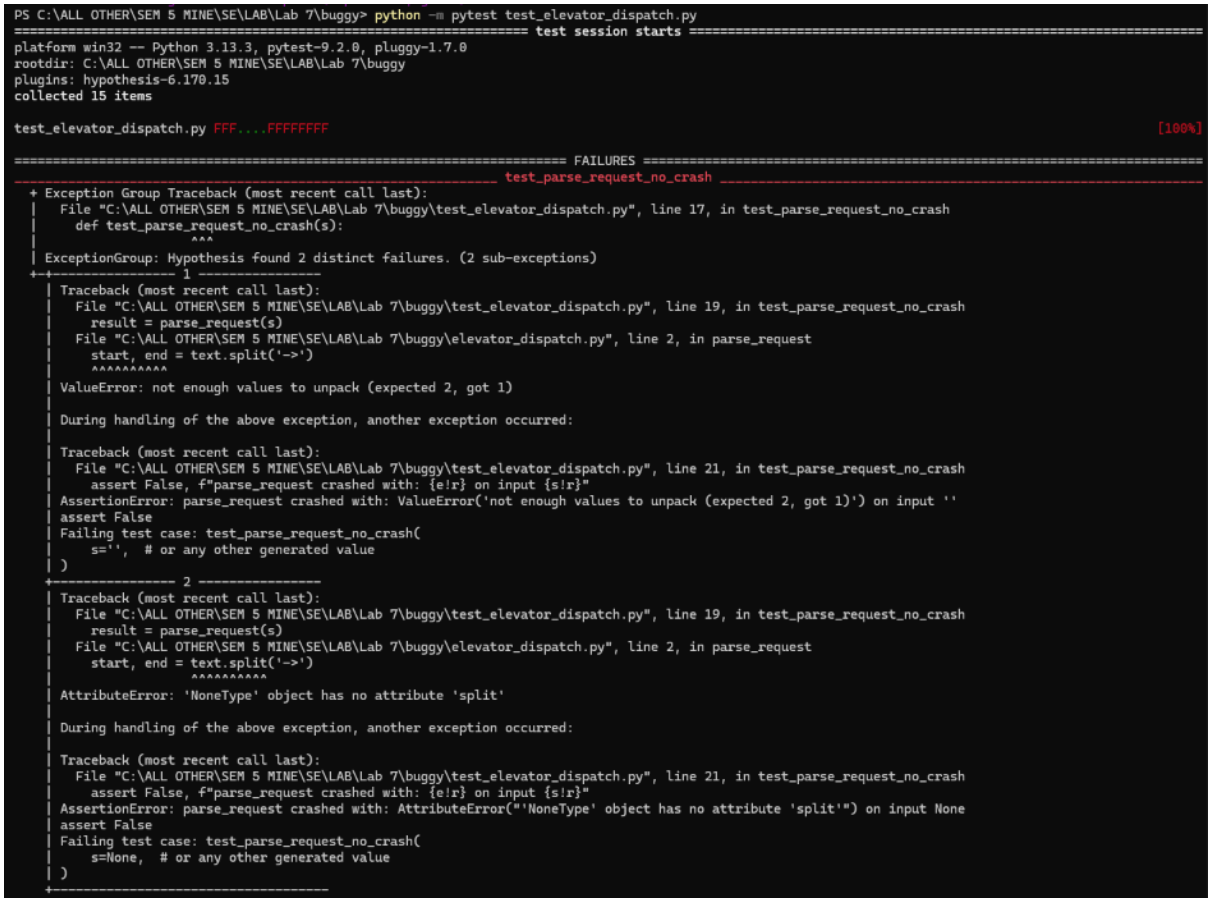
```
PS C:\ALL OTHER\SEM 5 MINE\SE\LAB\Lab 7\buggy> python elevator_dispatch.py
==== Running parse_request ====
Enter a request as FROM->TO (e.g. 5->9): 5->9
From and To: (5, 9)

==== Running total_travel ====
Enter floors visited as CSV (e.g. 1,5,3,9): 1,5,3,9
Floors Travelled: 12

==== Running load_percent ====
Enter load as 'X of Y' (e.g. 450 of 630): 450 of 630
Load Percent: 71
PS C:\ALL OTHER\SEM 5 MINE\SE\LAB\Lab 7\buggy> python elevator_dispatch.py
==== Running parse_request ====
Enter a request as FROM->TO (e.g. 5->9): 5-9
Traceback (most recent call last):
  File "C:\ALL OTHER\SEM 5 MINE\SE\LAB\Lab 7\buggy\elevator_dispatch.py", line 30, in <module>
    main()
    ~~~~^^
  File "C:\ALL OTHER\SEM 5 MINE\SE\LAB\Lab 7\buggy\elevator_dispatch.py", line 16, in main
    result = parse_request(user_input)
  File "C:\ALL OTHER\SEM 5 MINE\SE\LAB\Lab 7\buggy\elevator_dispatch.py", line 2, in parse_request
    start, end = text.split('->')
    ^^^^^^^^^^
ValueError: not enough values to unpack (expected 2, got 1)
```

Valid input works, but malformed input crashes the program: ValueError for a missing arrow or an empty CSV element, ZeroDivisionError for a capacity of 0.

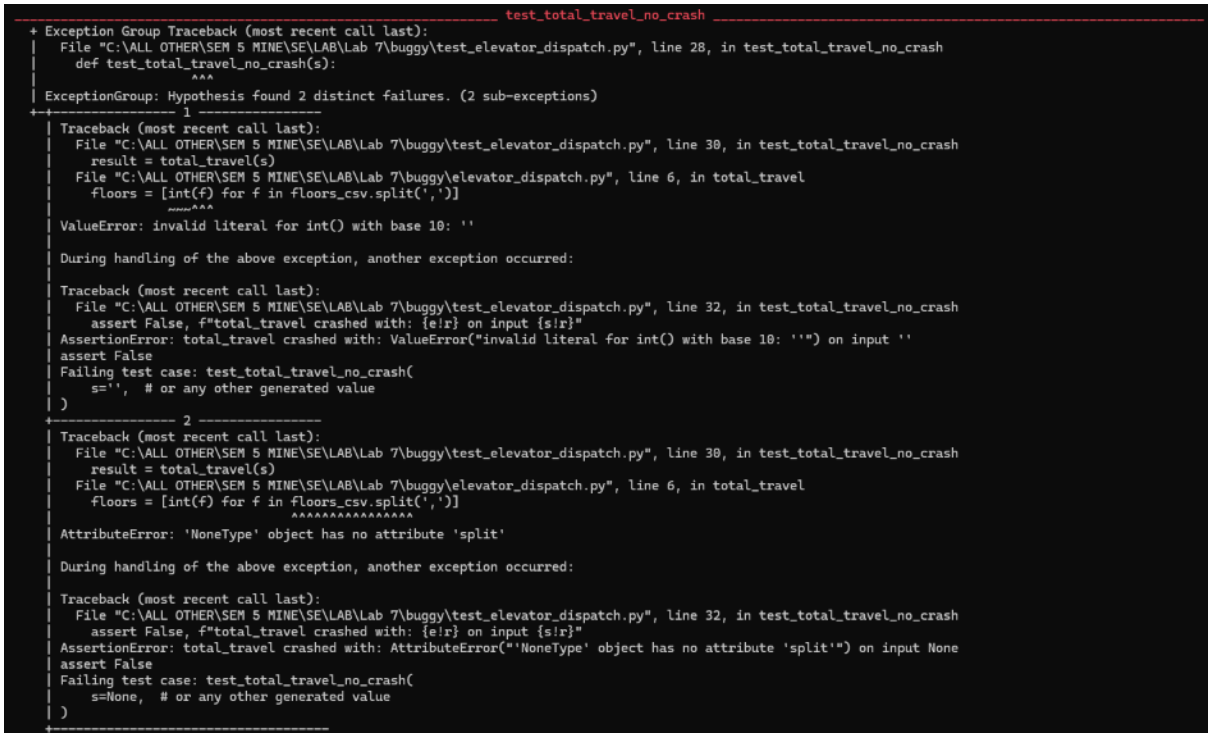
STEP 4: Failing Hypothesis fuzz tests (against the original code)



```
PS C:\ALL OTHER\SEM 5 MINE\SE\LAB\Lab 7\buggy> python -m pytest test_elevator_dispatch.py
============================= test session starts =============================
platform win32 -- Python 3.13.3, pytest-9.2.0, pluggy-1.7.0
rootdir: C:\ALL OTHER\SEM 5 MINE\SE\LAB\Lab 7\buggy
plugins: hypothesis-6.170.15
collected 15 items

test_elevator_dispatch.py FFF....FFFFFFFF                                [100%]

================================== FAILURES ===================================
_____________________________ test_parse_request_no_crash _____________________________
  + Exception Group Traceback (most recent call last):
  |   File "C:\ALL OTHER\SEM 5 MINE\SE\LAB\Lab 7\buggy\test_elevator_dispatch.py", line 17, in test_parse_request_no_crash
  |     def test_parse_request_no_crash(s):
  |                ^^^
  | ExceptionGroup: Hypothesis found 2 distinct failures. (2 sub-exceptions)
  +-+---------------- 1 ----------------
    | Traceback (most recent call last):
    |   File "C:\ALL OTHER\SEM 5 MINE\SE\LAB\Lab 7\buggy\test_elevator_dispatch.py", line 19, in test_parse_request_no_crash
    |     result = parse_request(s)
    |   File "C:\ALL OTHER\SEM 5 MINE\SE\LAB\Lab 7\buggy\elevator_dispatch.py", line 2, in parse_request
    |     start, end = text.split('->')
    |     ^^^^^^^^^^
    | ValueError: not enough values to unpack (expected 2, got 1)
    |
    | During handling of the above exception, another exception occurred:
    |
    | Traceback (most recent call last):
    |   File "C:\ALL OTHER\SEM 5 MINE\SE\LAB\Lab 7\buggy\test_elevator_dispatch.py", line 21, in test_parse_request_no_crash
    |     assert False, f"parse_request crashed with: {e!r} on input {s!r}"
    | AssertionError: parse_request crashed with: ValueError('not enough values to unpack (expected 2, got 1)') on input ''
    | assert False
    | Failing test case: test_parse_request_no_crash(
    |     s='',  # or any other generated value
    | )
    +---------------- 2 ----------------
    | Traceback (most recent call last):
    |   File "C:\ALL OTHER\SEM 5 MINE\SE\LAB\Lab 7\buggy\test_elevator_dispatch.py", line 19, in test_parse_request_no_crash
    |     result = parse_request(s)
    |   File "C:\ALL OTHER\SEM 5 MINE\SE\LAB\Lab 7\buggy\elevator_dispatch.py", line 2, in parse_request
    |     start, end = text.split('->')
    |                  ^^^^^^^^^^
    | AttributeError: 'NoneType' object has no attribute 'split'
    |
    | During handling of the above exception, another exception occurred:
    |
    | Traceback (most recent call last):
    |   File "C:\ALL OTHER\SEM 5 MINE\SE\LAB\Lab 7\buggy\test_elevator_dispatch.py", line 21, in test_parse_request_no_crash
    |     assert False, f"parse_request crashed with: {e!r} on input {s!r}"
    | AssertionError: parse_request crashed with: AttributeError("'NoneType' object has no attribute 'split'") on input None
    | assert False
    | Failing test case: test_parse_request_no_crash(
    |     s=None,  # or any other generated value
    | )
    +------------------------------------
```



```
_____________________________ test_total_travel_no_crash _____________________________
  + Exception Group Traceback (most recent call last):
  |   File "C:\ALL OTHER\SEM 5 MINE\SE\LAB\Lab 7\buggy\test_elevator_dispatch.py", line 28, in test_total_travel_no_crash
  |     def test_total_travel_no_crash(s):
  |                ^^^
  | ExceptionGroup: Hypothesis found 2 distinct failures. (2 sub-exceptions)
  +-+---------------- 1 ----------------
    | Traceback (most recent call last):
    |   File "C:\ALL OTHER\SEM 5 MINE\SE\LAB\Lab 7\buggy\test_elevator_dispatch.py", line 30, in test_total_travel_no_crash
    |     result = total_travel(s)
    |   File "C:\ALL OTHER\SEM 5 MINE\SE\LAB\Lab 7\buggy\elevator_dispatch.py", line 6, in total_travel
    |     floors = [int(f) for f in floors_csv.split(',')]
    |               ~~~^^^
    | ValueError: invalid literal for int() with base 10: ''
    |
    | During handling of the above exception, another exception occurred:
    |
    | Traceback (most recent call last):
    |   File "C:\ALL OTHER\SEM 5 MINE\SE\LAB\Lab 7\buggy\test_elevator_dispatch.py", line 32, in test_total_travel_no_crash
    |     assert False, f"total_travel crashed with: {e!r} on input {s!r}"
    | AssertionError: total_travel crashed with: ValueError("invalid literal for int() with base 10: ''") on input ''
    | assert False
    | Failing test case: test_total_travel_no_crash(
    |     s='',  # or any other generated value
    | )
    +---------------- 2 ----------------
    | Traceback (most recent call last):
    |   File "C:\ALL OTHER\SEM 5 MINE\SE\LAB\Lab 7\buggy\test_elevator_dispatch.py", line 30, in test_total_travel_no_crash
    |     result = total_travel(s)
    |   File "C:\ALL OTHER\SEM 5 MINE\SE\LAB\Lab 7\buggy\elevator_dispatch.py", line 6, in total_travel
    |     floors = [int(f) for f in floors_csv.split(',')]
    |                              ^^^^^^^^^^^^^^^^
    | AttributeError: 'NoneType' object has no attribute 'split'
    |
    | During handling of the above exception, another exception occurred:
    |
    | Traceback (most recent call last):
    |   File "C:\ALL OTHER\SEM 5 MINE\SE\LAB\Lab 7\buggy\test_elevator_dispatch.py", line 32, in test_total_travel_no_crash
    |     assert False, f"total_travel crashed with: {e!r} on input {s!r}"
    | AssertionError: total_travel crashed with: AttributeError("'NoneType' object has no attribute 'split'") on input None
    | assert False
    | Failing test case: test_total_travel_no_crash(
    |     s=None,  # or any other generated value
    | )
    +------------------------------------
```

```
_____ test_load_percent_no_crash _____
  + Exception Group Traceback (most recent call last):
  |   File "C:\ALL OTHER\SEM 5 MINE\SE\LAB\Lab 7\buggy\test_elevator_dispatch.py", line 37, in test_load_percent_no_crash
  |     def test_load_percent_no_crash(s):
  |                ^^^
  | ExceptionGroup: Hypothesis found 2 distinct failures. (2 sub-exceptions)
  +-+---------------- 1 ----------------
    | Traceback (most recent call last):
    |   File "C:\ALL OTHER\SEM 5 MINE\SE\LAB\Lab 7\buggy\test_elevator_dispatch.py", line 39, in test_load_percent_no_crash
    |     result = load_percent(s)
    |   File "C:\ALL OTHER\SEM 5 MINE\SE\LAB\Lab 7\buggy\elevator_dispatch.py", line 10, in load_percent
    |     load, capacity = text.split(' of ')
    |     ^^^^^^^^^^^^^^
    | ValueError: not enough values to unpack (expected 2, got 1)
    |
    | During handling of the above exception, another exception occurred:
    |
    | Traceback (most recent call last):
    |   File "C:\ALL OTHER\SEM 5 MINE\SE\LAB\Lab 7\buggy\test_elevator_dispatch.py", line 41, in test_load_percent_no_crash
    |     assert False, f"load_percent crashed with: {e!r} on input {s!r}"
    | AssertionError: load_percent crashed with: ValueError('not enough values to unpack (expected 2, got 1)') on input ''
    | assert False
    | Failing test case: test_load_percent_no_crash(
    |     s='',  # or any other generated value
    | )
    +---------------- 2 ----------------
    | Traceback (most recent call last):
    |   File "C:\ALL OTHER\SEM 5 MINE\SE\LAB\Lab 7\buggy\test_elevator_dispatch.py", line 39, in test_load_percent_no_crash
    |     result = load_percent(s)
    |   File "C:\ALL OTHER\SEM 5 MINE\SE\LAB\Lab 7\buggy\elevator_dispatch.py", line 10, in load_percent
    |     load, capacity = text.split(' of ')
    |                      ^^^^^^^^^^
    | AttributeError: 'NoneType' object has no attribute 'split'
    |
    | During handling of the above exception, another exception occurred:
    |
    | Traceback (most recent call last):
    |   File "C:\ALL OTHER\SEM 5 MINE\SE\LAB\Lab 7\buggy\test_elevator_dispatch.py", line 41, in test_load_percent_no_crash
    |     assert False, f"load_percent crashed with: {e!r} on input {s!r}"
    | AssertionError: load_percent crashed with: AttributeError("'NoneType' object has no attribute 'split'") on input None
    | assert False
    | Failing test case: test_load_percent_no_crash(
    |     s=None,  # or any other generated value
    | )
    +------------------------------------
```

```
_____ test_load_percent_rejects_zero_or_negative_capacity _____
  + Exception Group Traceback (most recent call last):
  |   File "C:\ALL OTHER\SEM 5 MINE\SE\LAB\Lab 7\buggy\test_elevator_dispatch.py", line 73, in test_load_percent_rejects_zero_or_negative_capacity
  |     def test_load_percent_rejects_zero_or_negative_capacity(load, capacity):
  |                ^^^
  | ExceptionGroup: Hypothesis found 2 distinct failures. (2 sub-exceptions)
  +-+---------------- 1 ----------------
    | Traceback (most recent call last):
    |   File "C:\ALL OTHER\SEM 5 MINE\SE\LAB\Lab 7\buggy\test_elevator_dispatch.py", line 74, in test_load_percent_rejects_zero_or_negative_capacity
    |     assert load_percent(f"{load} of {capacity}") is None      # was ZeroDivisionError / negative %
    |     ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
    | AssertionError: assert 0 is None
    |  +  where 0 = load_percent('0 of -1')
    | Failing test case: test_load_percent_rejects_zero_or_negative_capacity(
    |     # The test always failed when commented parts were varied together.
    |     load=0,  # or any other generated value
    |     capacity=-1,  # or any other generated value
    | )
    +---------------- 2 ----------------
    | Traceback (most recent call last):
    |   File "C:\ALL OTHER\SEM 5 MINE\SE\LAB\Lab 7\buggy\test_elevator_dispatch.py", line 74, in test_load_percent_rejects_zero_or_negative_capacity
    |     assert load_percent(f"{load} of {capacity}") is None      # was ZeroDivisionError / negative %
    |            ~~~~~~~~~~~~~~~~^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
    |   File "C:\ALL OTHER\SEM 5 MINE\SE\LAB\Lab 7\buggy\elevator_dispatch.py", line 11, in load_percent
    |     return int(load) * 100 // int(capacity)
    |            ~~~~~~~~~~~~~~~~~~~~~~~~^^~~~~~~~~~~~~~~~~~
    | ZeroDivisionError: integer division or modulo by zero
    | Failing test case: test_load_percent_rejects_zero_or_negative_capacity(
    |     # The test always failed when commented parts were varied together.
    |     load=0,  # or any other generated value
    |     capacity=0,  # or any other generated value
    | )
    +------------------------------------
_____ test_load_percent_rejects_negative_load _____

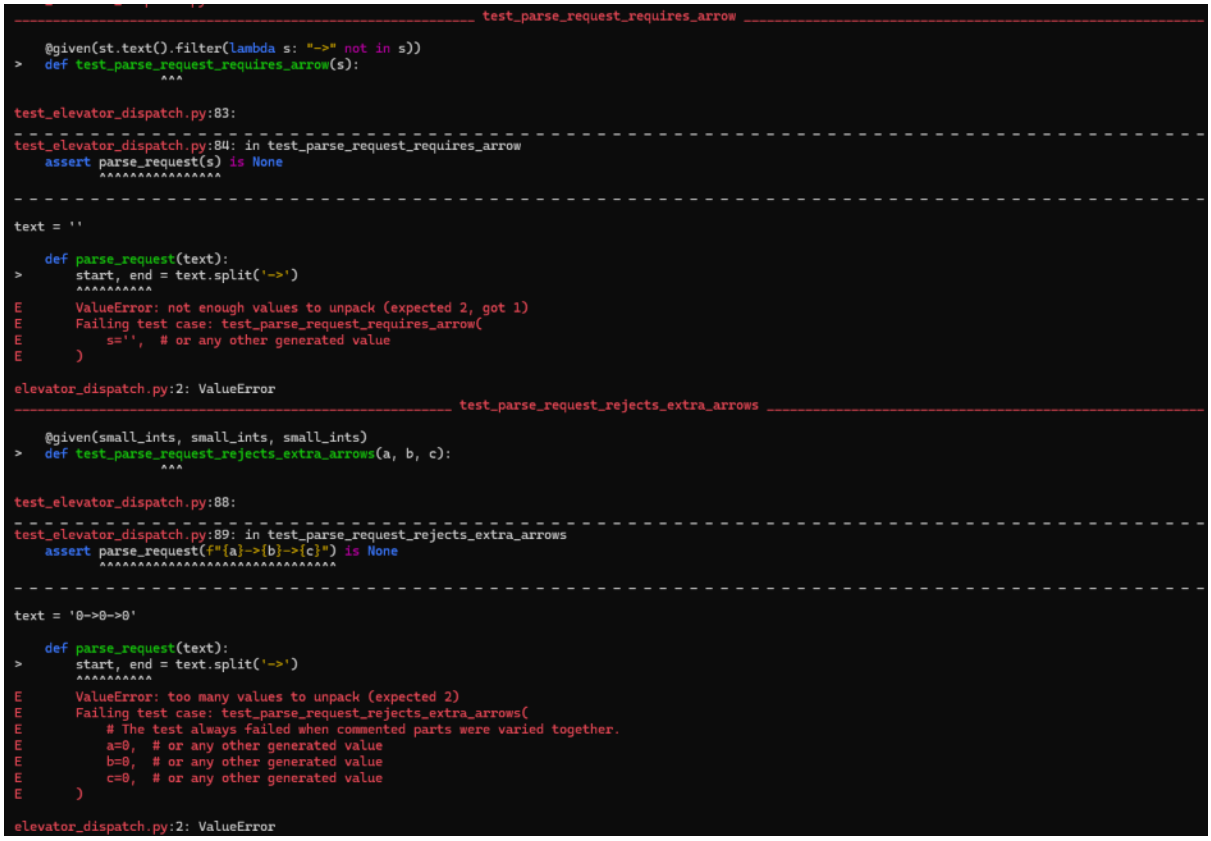
    @given(st.integers(min_value=-999_999_999, max_value=-1), st.integers(min_value=1, max_value=10**6))
>   def test_load_percent_rejects_negative_load(load, capacity):
                ^^^

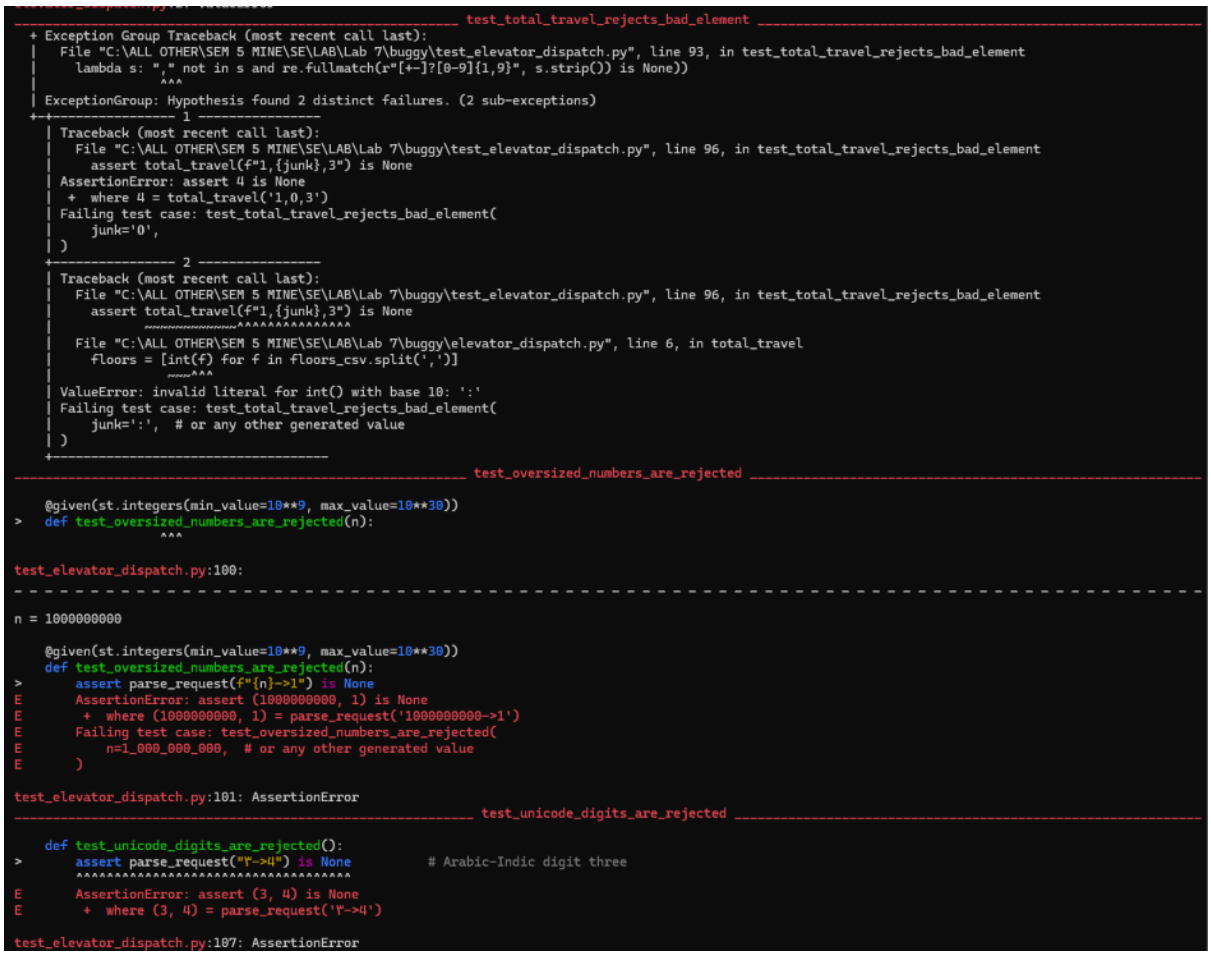
test_elevator_dispatch.py:78:
- - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - -

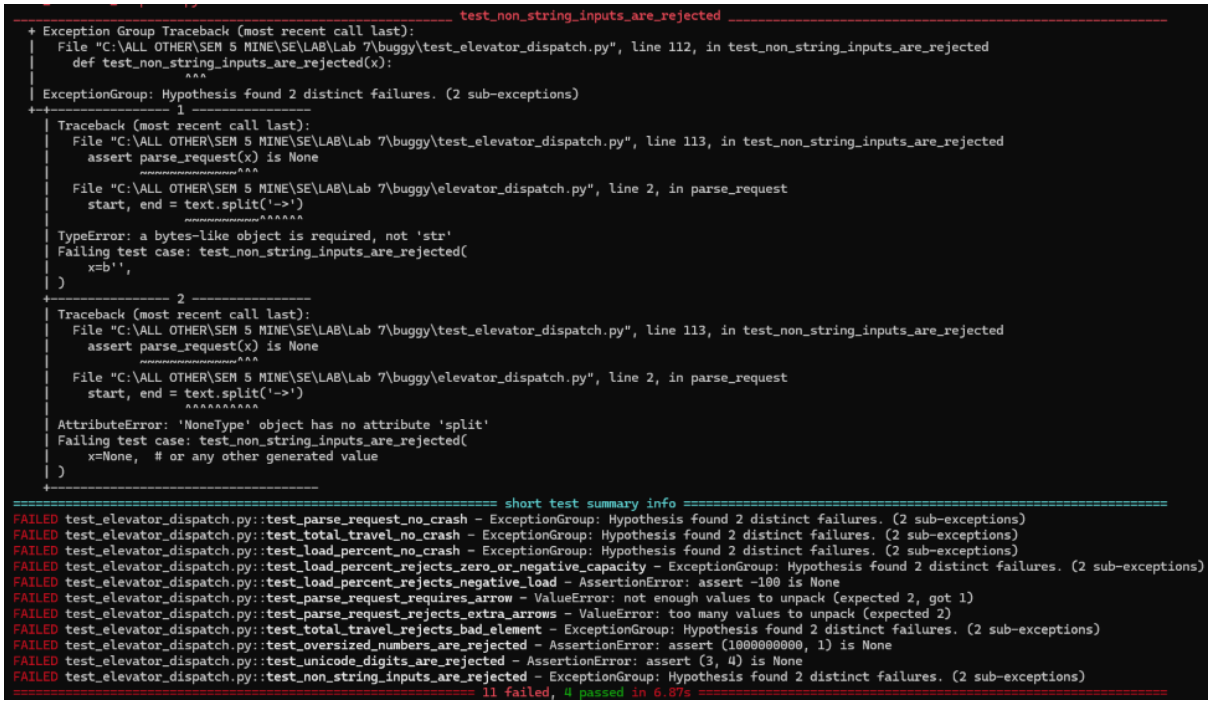
load = -1, capacity = 1

    @given(st.integers(min_value=-999_999_999, max_value=-1), st.integers(min_value=1, max_value=10**6))
    def test_load_percent_rejects_negative_load(load, capacity):
>       assert load_percent(f"{load} of {capacity}") is None
E       AssertionError: assert -100 is None
E        +  where -100 = load_percent('-1 of 1')
E       Failing test case: test_load_percent_rejects_negative_load(
E           # The test always failed when commented parts were varied together.
E           load=-1,  # or any other generated value
E           capacity=1,  # or any other generated value
E       )

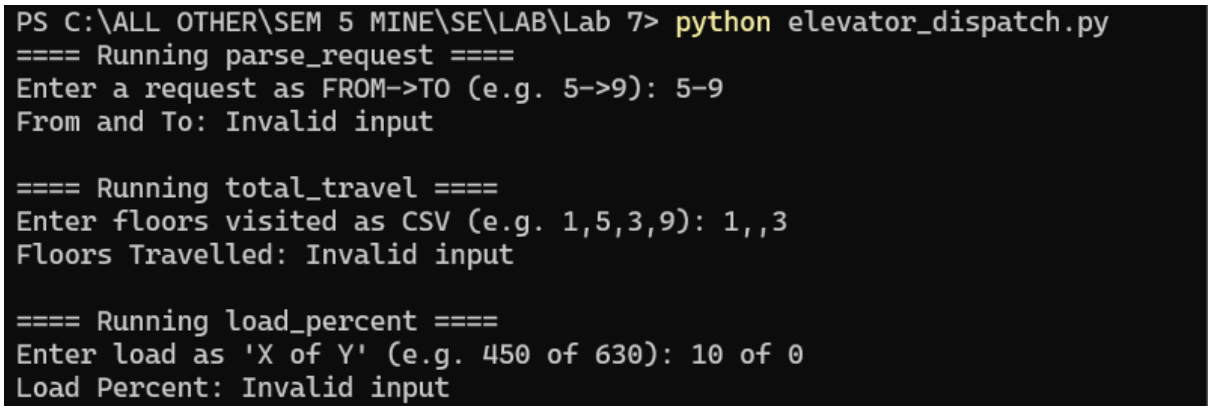
test_elevator_dispatch.py:79: AssertionError
```


test_parse_request_requires_arrow
@given(st.text().filter(lambda s: "->" not in s))
def test_parse_request_requires_arrow(s):
test_elevator_dispatch.py:83:
test_elevator_dispatch.py:84: in test_parse_request_requires_arrow
assert parse_request(s) is None
text = ''
def parse_request(text):
start, end = text.split('->')
ValueError: not enough values to unpack (expected 2, got 1)
Failing test case: test_parse_request_requires_arrow(
s='', # or any other generated value
)
elevator_dispatch.py:2: ValueError
test_parse_request_rejects_extra_arrows
@given(small_ints, small_ints, small_ints)
def test_parse_request_rejects_extra_arrows(a, b, c):
test_elevator_dispatch.py:88:
test_elevator_dispatch.py:89: in test_parse_request_rejects_extra_arrows
assert parse_request(f"{a}->{b}->{c}") is None
text = '0->0->0'
def parse_request(text):
start, end = text.split('->')
ValueError: too many values to unpack (expected 2)
Failing test case: test_parse_request_rejects_extra_arrows(
The test always failed when commented parts were varied together.
a=0, # or any other generated value
b=0, # or any other generated value
c=0, # or any other generated value
)
elevator_dispatch.py:2: ValueError


test_total_travel_rejects_bad_element
+ Exception Group Traceback (most recent call last):
File "C:\ALL OTHER\SEM 5 MINE\SE\LAB\Lab 7\buggy\test_elevator_dispatch.py", line 93, in test_total_travel_rejects_bad_element
lambda s: "," not in s and re.fullmatch(r"[+-]?[0-9]{1,9}", s.strip()) is None))
ExceptionGroup: Hypothesis found 2 distinct failures. (2 sub-exceptions)
+---------- 1 ----------
Traceback (most recent call last):
File "C:\ALL OTHER\SEM 5 MINE\SE\LAB\Lab 7\buggy\test_elevator_dispatch.py", line 96, in test_total_travel_rejects_bad_element
assert total_travel(f"1,{junk},3") is None
AssertionError: assert 4 is None
+ where 4 = total_travel('1,0,3')
Failing test case: test_total_travel_rejects_bad_element(
junk='0',
)
+---------- 2 ----------
Traceback (most recent call last):
File "C:\ALL OTHER\SEM 5 MINE\SE\LAB\Lab 7\buggy\test_elevator_dispatch.py", line 96, in test_total_travel_rejects_bad_element
assert total_travel(f"1,{junk},3") is None
File "C:\ALL OTHER\SEM 5 MINE\SE\LAB\Lab 7\buggy\elevator_dispatch.py", line 6, in total_travel
floors = [int(f) for f in floors_csv.split(',')]
ValueError: invalid literal for int() with base 10: ':'
Failing test case: test_total_travel_rejects_bad_element(
junk=':', # or any other generated value
)
test_oversized_numbers_are_rejected
@given(st.integers(min_value=10**9, max_value=10**30))
def test_oversized_numbers_are_rejected(n):
test_elevator_dispatch.py:100:
n = 1000000000
@given(st.integers(min_value=10**9, max_value=10**30))
def test_oversized_numbers_are_rejected(n):
assert parse_request(f"{n}->1") is None
AssertionError: assert (1000000000, 1) is None
+ where (1000000000, 1) = parse_request('1000000000->1')
Failing test case: test_oversized_numbers_are_rejected(
n=1_000_000_000, # or any other generated value
)
test_elevator_dispatch.py:101: AssertionError
test_unicode_digits_are_rejected
def test_unicode_digits_are_rejected():
assert parse_request("٣->4") is None   # Arabic-Indic digit three
AssertionError: assert (3, 4) is None
+ where (3, 4) = parse_request('٣->4')
test_elevator_dispatch.py:107: AssertionError



STEP 5: Execution of the corrected application



STEP 6: Final test run against the corrected application

